- *Image embeddings*—These are like what word embeddings are for words. They are numerical representations of images. When visualized, similar images will be close together in the embedding space. For example, images of dogs would be near each other, while images of cars would be in a different cluster.
- *Graph embeddings*—These are optimized data structures that are used for capturing the relationships between nodes. When visualized, nodes that are well connected in the graph will be close together in the embedding space. For example, in a social network graph, users with many mutual friends would be embedded closer together.

In practical terms, it's important to associate embeddings with lists of floating-point numbers. To the human eye, these numbers may not make any sense, but to machine learning models, they are powerful representations of complex information. This brings us to the following clarification.

EMBEDDINGS ARE NOT LIKE MACHINE LANGUAGE IN COMPUTERS

Don't get confused by thinking that embeddings are a language for AI models. They are fundamentally different in terms of their purpose, functionality, and level of abstraction.

Machine language, also known as machine code, is the lowest-level programming language that is directly understood by a computer's CPU. It consists of binary code (0s and 1s) that represents instructions for the CPU to execute. It is specific to hardware architecture, and it is not human readable.

Embeddings, on the other hand, are numerical representations of data (such as words, sentences, or images) in a continuous vector space; they are also not human readable. They are used in machine learning to capture semantic meaning and relationships between different pieces of data. The AI model does not use them for following instructions in a procedural manner. Instead, they serve as input data for machine learning models, which can then perform tasks such as classification, clustering, or recommendation. This distinction is important when working with different embedding schemes at different levels of abstraction.

EMBEDDINGS ARE TIED TO A PARTICULAR MODEL

In the current state, embeddings are highly specific to the architecture and training of the AI models that generate them. As a result, embeddings produced by one model may not be compatible with those from another model.

Mixing and matching embeddings from different models arbitrarily can lead to inaccurate or misleading results, as the semantic meanings captured by the embeddings may differ significantly. For instance, embeddings generated by OpenAI may not match with those from a different provider, such as a local model or another AI provider. This, of course, can create issues when storing embeddings in systems that cater to multiple AI models, particularly when having to cater for increased storage and versioning requirements.

Therefore, it is important to develop a system that uses embeddings from the same or compatible model for tasks such as similarity comparisons, clustering, or input to